GSoC 2026 Evaluation Exercise Report

Author: Somnath Karmakar (somnathkarmakar1203@gmail.com)
Applying for Project: Parallel Processing Improvements in Julia Jet Reconstruction (HSF/CERN)

Table of Contents

1. **Benchmarking Methodology**
2. **Serial Version Analysis**
    - Baseline Performance (serial-euclid.jl)
    - Improved Serial Version (improved-serial.jl)
3. **Parallelisation Strategy (parallel-euclid.jl)**
4. **Hardware Scaling & Benchmarking Results**
    - Performance Plots
    - Commentary on Results
5. **Reproduction Instructions**
6. **Discussion on GPU porting**
    - The PCIe Bottleneck and Memory Streaming
        - Asynchronous Streams
        - Unified Memory
    - Memory Coalescing and SoA
        - The SoA Advantage
        - Caveat
    - Shared Memory Tiling and Collaborative Loading
        - Collaborative Loading
        - Synchronization
    - Warp Divergence and Branchless Math
        - Numerical Stability
    - A Bonus Point: H100 and Beyond
        - HBM3e Bandwidth
        - TMA (Tensor Memory Accelerator)

- **Conclusion**

1. **AI Usage Statement**

1. Benchmarking Methodology

To ensure accurate, reproducible performance metrics, the benchmarking was conducted using Julia's `BenchmarkTools.jl` package, adhering to the following best practices:

1. **JIT Compilation and Warm-ups:** Julia utilizes a Just-In-Time (JIT) compiler. If a function is timed on its very first execution, the result includes the compilation overhead. To isolate pure execution time, a warm-up run of every function was executed prior to benchmarking.
2. **Variable Interpolation:** Global variables passed into the benchmarking macros were interpolated (e.g., `@benchmark pairwise_distances($points)`). This prevents artificial memory allocation overhead caused by Julia boxing global variables to ensure type stability.
3. **Statistical Stability:** Execution times can fluctuate due to OS background tasks. The `@benchmark` macro executes the target function several times. Following standard practice, the **minimum** execution time `(minimum(trial))` was extracted, as it represents the code running with the least amount of OS-induced noise. The minimum execution time approximates the performance when the system experiences minimal OS interference, making it a better estimate of the true computational cost.

2. Serial Version Analysis

Baseline Performance (serial-euclid.jl)

The initial, naive serial implementation was benchmarked without any changes:

- **Execution Time:** 670-690 ms
- **Allocations:** 3 allocations (381.47 MiB)

Efficiency Critique:
The baseline code is mathematically correct but computationally inefficient for several reasons:

1. **Algorithmic Inefficiency:** It iterates over an N x N grid, calculating both d(A, B) and d(B, A). Because Euclidean distance is symmetric, this results in 100 million distance calculations when only around 50 million (49,995,000 to be precise) are strictly necessary.
2. **Bounds Checking Overhead:** The standard for loops trigger Julia's safety bounds-checking on every array access, creating significant overhead inside the innermost loop.
3. **Memory Layout (AoS):** The 10,000 x 3 input is an Array of Structures (AoS). Because Julia arrays are column-major, accessing x, y, z coordinates for a single point requires

jumping across memory, resulting in frequent CPU cache misses. The Nx3 matrix leads to strided memory access when iterating over points, which reduces cache efficiency.

Improved Serial Version (improved-serial.jl)

To address these bottlenecks before introducing parallelism, an improved serial version was developed.

- **Execution Time:** 390 - 410 ms *(~40% speedup over baseline on average)*

Key Improvements:

- **Triangular Loop:** Modified the inner loop to `for j in i+1:n`, exploiting matrix symmetry to cut the computational workload in half.
- **Extracting the columns first:** Adheres to Julia's column-major order as the naive loop needed non-sequential memory fetches due this fact, which slows things down.
- **Disabling Bounds Checks:** Applied the `@inbounds` macro to bypass runtime safety checks.
- **Local Caching:** Cached the coordinates of point i (`xi`, `yi`, `zi`) outside the inner loop to prevent fetching them 10,000 times redundantly.

3. Parallelisation Strategy (parallel-euclid.jl)

This is a task of data parallelization. To scale the performance across a multi-core CPU, the parallel architecture was designed around memory locality and vectorization.

1. Data Layout & SIMD (Structure of Arrays)
The N x 3 input array was explicitly transposed into a 3 x N matrix `(collect(points'))`. This converted the data into a Structure of Arrays (SoA), each coordinate array (x, y, z) becomes contiguous, enabling vector loads.. This is a strict prerequisite for **SIMD (Single Instruction, Multiple Data)** vectorization, allowing the CPU to load multiple coordinates into wide AVX registers simultaneously.

2. Cache Blocking (Tiling)
To prevent the CPU from stalling while waiting for RAM, the distance matrix calculation was partitioned into 64 x 64 blocks. 64 points × 3 floats ≈ 1536 bytes - fits comfortably in L1/L2 cache. This ensures the active working set of points fits entirely within the ultra-fast L1/L2 CPU cache. For the blocks on the diagonal for which the real math is ½ of the block, we should treat it just like a non-diagonal block and calculate all through 64x64 points because conditional logic adds overhead that overweighs the very small amount of redundant calculations needed to be done on a large 10000x10000 dataset of points utilizing multiprocessing.

3. Load Balancing & Task Flattening

Because the algorithm only calculates the lower triangle of the matrix, assigning threads directly to rows would cause severe triangular imbalance (Thread 1 would do drastically more work than the final thread). To solve this, the block coordinates were flattened into a 1D tasks list. Utilizing threading (`Threads.@threads` over the task list) now ensures that threads divide the work efficiently, keeping resource utilization near 100%.

4. Micro-Optimization & Zero-Cost Allocation

Inside the blocks, the `@turbo` macro from `LoopVectorization.jl` was utilized to aggressively unroll the inner loops and force SIMD Fused-Multiply-Add (FMA) instructions. Additionally, the 10,000 x 10,000 distance matrix was allocated using `Matrix{undef}`, saving the OS from writing ~381 MiB of zeros prior to the calculation. A final, dedicated multi-threaded pass was used to symmetrize the matrix contiguously.

5. SIMD processes math just a tad bit differently

```
julia> naive_result = pairwise_distances(points); parallel_result = pairwise_distances_parallel(points); naive_result ≈ parallel_result
true
```

```
julia> naive_result - parallel_result
10000×10000 Matrix{Float32}:
 0.0          0.0          0.0          0.0          0.0          0.0          0.0          …  0.0          0.0          0.0          0.0          0.0          0.0          0.0
 0.0          0.0          0.0          0.0          0.0          0.0          0.0             0.0          0.0          0.0          0.0          0.0          0.0          0.0
 0.0          0.0          0.0          2.98023f-8   0.0          0.0          0.0             0.0          0.0         -2.98023f-8   0.0          0.0          0.0          0.0
 0.0          0.0          2.98023f-8   0.0         -5.96046f-8   0.0          0.0             0.0          0.0          0.0          1.49012f-8   0.0          5.96046f-8   0.0
 0.0          0.0          0.0         -5.96046f-8   0.0          0.0          0.0             0.0         -2.98023f-8   0.0          0.0          0.0          0.0          0.0
 0.0          0.0          0.0          0.0          0.0          0.0          0.0          …  0.0          0.0          0.0          0.0          0.0          0.0         -1.19209f-7
 0.0          0.0          0.0          0.0          0.0          0.0          0.0             0.0          0.0          0.0          0.0          0.0          0.0          0.0
 0.0          0.0          0.0          0.0          0.0          0.0          0.0             0.0          0.0          0.0          0.0          0.0          0.0          0.0
-2.98023f-8   0.0          0.0          0.0          5.96046f-8   0.0          0.0             0.0          0.0          0.0          0.0          0.0          0.0          0.0
 0.0          0.0          0.0          0.0          0.0          0.0          0.0            -5.96046f-8   0.0          5.96046f-8   0.0          0.0          0.0          0.0
 0.0          0.0          0.0          0.0          0.0          0.0          0.0          …  0.0          0.0          0.0          0.0          0.0          0.0          0.0
 0.0          0.0          0.0          0.0          0.0          0.0         -1.19209f-7      0.0          0.0          0.0          0.0          5.96046f-8   0.0          0.0
 5.96046f-8   0.0          0.0          0.0          5.96046f-8   7.45058f-9   0.0             0.0          0.0          0.0          0.0          0.0          0.0          0.0
 0.0          0.0          0.0          0.0          0.0         -1.19209f-7   0.0            -5.96046f-8   0.0          0.0         -5.96046f-8   0.0          0.0          0.0
 0.0          0.0          0.0          0.0          0.0          0.0          0.0             0.0          0.0          0.0          0.0          0.0         -5.96046f-8
 0.0          0.0          0.0          0.0          0.0          0.0          0.0          …  0.0          5.96046f-8   0.0          0.0         -1.49012f-8   0.0          0.0
 0.0          2.98023f-8   0.0          0.0          0.0          0.0         -5.96046f-8      2.98023f-8   0.0          0.0         -5.96046f-8   0.0          0.0          0.0
 0.0          0.0          0.0          0.0          0.0          0.0          5.96046f-8      0.0          0.0          2.98023f-8   0.0          0.0          0.0          0.0
 0.0          0.0          0.0          0.0          2.98023f-8   0.0          0.0            -2.98023f-8  -5.96046f-8   2.98023f-8   0.0          0.0         -2.98023f-8   5.96046f-8
 0.0          0.0          0.0          0.0          0.0         -5.96046f-8   1.19209f-7      5.96046f-8   1.19209f-7  -5.96046f-8   0.0          0.0          0.0          0.0
 ⋮                                                               ⋮                         ⋱                            ⋮
 5.96046f-8  -1.19209f-7   0.0          0.0          5.96046f-8   0.0          0.0          …  0.0          0.0          0.0          1.49012f-8   0.0          0.0          0.0
 0.0          0.0          0.0          0.0          0.0          0.0          0.0             0.0          0.0          0.0          0.0          0.0          0.0          0.0
 0.0          5.96046f-8   0.0          0.0          0.0         -5.96046f-8   0.0             0.0          0.0          0.0          5.96046f-8   0.0          0.0          0.0
 5.96046f-8  -2.98023f-8   0.0          0.0         -2.98023f-8   0.0          0.0             5.96046f-8   0.0          0.0          0.0          0.0          5.96046f-8   0.0
 0.0          0.0          0.0          0.0          0.0          0.0          0.0             0.0          0.0          0.0          0.0          0.0          0.0          0.0
 0.0          0.0          0.0          0.0          0.0          0.0          0.0          …  0.0          0.0          0.0          0.0          0.0          0.0          0.0
 0.0          0.0          0.0          0.0          0.0          0.0          0.0             0.0          0.0          0.0          0.0          0.0          0.0          0.0
 0.0         -5.96046f-8   0.0          0.0          0.0          0.0         -5.96046f-8      0.0          0.0          0.0          0.0         -1.49012f-8   0.0          0.0
 5.96046f-8   0.0          0.0          0.0          0.0          0.0          0.0             0.0          0.0          0.0          0.0          0.0         -2.98023f-8
 0.0          0.0          0.0          0.0          0.0          0.0          0.0             0.0          0.0          0.0          0.0          2.98023f-8   0.0          0.0
 0.0          0.0          0.0          0.0          0.0          0.0          0.0          …  0.0          0.0          0.0          0.0          0.0          0.0          0.0
 0.0          0.0          0.0          0.0         -5.96046f-8   0.0          0.0             2.98023f-8   0.0          0.0          0.0          0.0          0.0          0.0
 0.0          0.0          0.0          5.96046f-8  -5.96046f-8   0.0          0.0             0.0          1.19209f-7   0.0          0.0          0.0          0.0          0.0
 0.0          0.0          0.0          0.0          0.0          0.0          0.0             0.0          0.0          0.0          0.0          0.0          0.0         -2.98023f-8
 0.0          0.0          0.0          0.0         -2.98023f-8   0.0          0.0             0.0          0.0          5.96046f-8   0.0          5.96046f-8   0.0          0.0
 0.0          0.0         -2.98023f-8   0.0          0.0          0.0          0.0          …  0.0          5.96046f-8   0.0          0.0          0.0          0.0          0.0
 0.0          0.0          0.0          1.49012f-8   0.0          0.0          0.0             0.0          0.0          0.0          0.0          0.0         -5.96046f-8   2.98023f-8
 0.0          0.0          0.0          0.0          0.0          0.0          0.0             0.0          5.96046f-8   0.0          0.0          0.0          0.0          0.0
 0.0          0.0          0.0          5.96046f-8   0.0          0.0          0.0             0.0          0.0          0.0         -5.96046f-8   0.0          0.0          0.0
 0.0          0.0          0.0          0.0          0.0         -1.19209f-7   0.0            -2.98023f-8   0.0          0.0          2.98023f-8   0.0          0.0          0.0

julia>
```

Due to floating point precision issues, there are infinitesimal differences between the matrices created by the serial function and the parallel version, but that is expected and is of no great concern since the matrices are otherwise equal as is evident in the output of the `\approx` comparison.

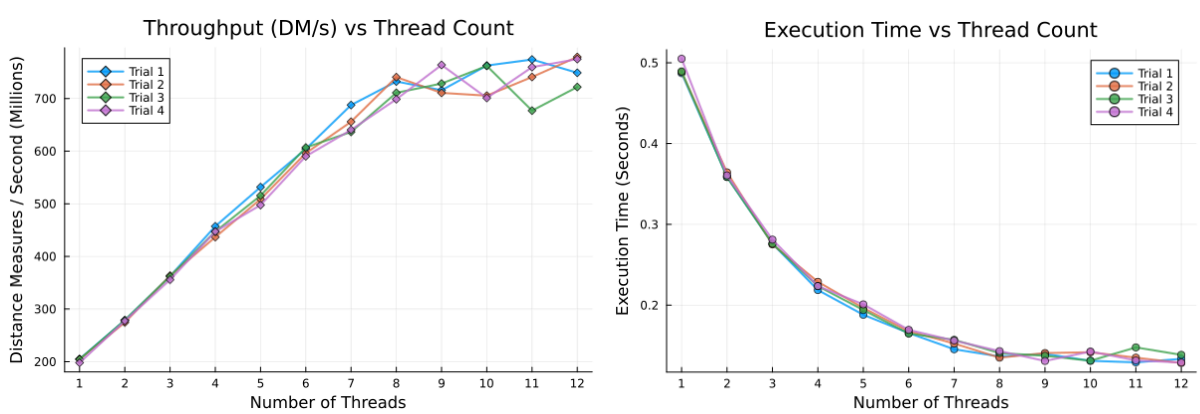
```
radon@RADIUM:/media/radon/DataA/JetRecoParallelGSoC$ julia -t 12 benchmark.jl
Starting calculation
Finished calculation

Benchmarks:

==== BASELINE SERIAL ====
  659.069 ms (3 allocations: 381.47 MiB)
  Throughput: 151.73 Million DM/s

==== IMPROVED SERIAL ====
  399.738 ms (9 allocations: 381.59 MiB)
  Throughput: 250.16 Million DM/s

==== PARALLEL ====
  122.432 ms (146 allocations: 381.98 MiB)
  Throughput: 816.78 Million DM/s
```

(General benchmarks comparing the original code, improved serial and parallel implementations)

4. Hardware Scaling & Benchmarking Results

To evaluate thread scaling, four separate trials were conducted, varying the active threads from 1 to 12 on an AMD Ryzen 5 4600H machine. Throughput was measured in **Distance Measures per Second (DM/s)**, calculated by dividing the total effective theoretical output (10^8 measures) by the execution time.

Performance Plots



(Images generated via plot_run_data.jl)

Commentary on Results

1. The Linear Scaling Region (1 to 8 Threads):
The algorithm demonstrates strong near-linear scaling (DPMS vs cores) up to 7-8 threads. In this region, each logical thread maps directly to a physical CPU core, allowing parallel execution of the `@turbo` SIMD instructions with minimal hardware contention.

2. The Hyper-Threading Plateau (6 to 12 Threads):
Beyond 7-8 threads, the scaling curve visibly flattens, exhibiting diminishing marginal returns. This inflection point corresponds directly to the physical hardware architecture: a typical hexacore processor possesses 6 physical cores but 12 logical threads. Because SIMD vectorization heavily utilizes the physical Floating Point Unit (FPU) of each core, two logical threads sharing the same physical core cannot execute heavy SIMD math instructions simultaneously; they must timeshare the FPU.

3. Memory Bandwidth Saturation:
The peak throughput of **~779 Million DM/s** (at ~0.128 seconds, i.e. a speedup of ~81%) indicates that the system has transitioned from a compute-bound state to a memory-bandwidth-bound state. The CPU's arithmetic throughput outpaces the rate at which the DDR memory controller can fetch the coordinate arrays and write back the resulting 381 MiB distance matrix.

5. Reproduction Instructions

To reproduce these results:

1. Ensure Julia is installed. Run `using Pkg; Pkg.add(["BenchmarkTools", "LoopVectorization", "Plots"])` in the REPL to ensure necessary packages are available.
2. Clone this repository and navigate to the root directory.
3. Run the automated bash script to generate raw scaling data: `sh ./run-parallel-benchmark.sh`
4. To run the full benchmark suite comparing Serial vs Parallel, execute: `julia -t auto benchmark.jl`
5. Replace `auto` in the above command with the number of threads you want to run on if you want a specific number.
6. Run `julia -t <number of cores | auto> parallel-benchmark.jl` if you want to benchmark the parallel code only once with the specified number of cores.

6. Discussion on GPU porting

While the current parallel CPU implementation utilizes many GPU-friendly concepts like **Structure of Arrays (SoA)** and **Tiling**, a production-grade GPU port must address specific hardware bottlenecks, most notably the PCIe bus and warp-level execution constraints.

1. The PCIe Bottleneck and Memory Streaming

The primary performance killer in GPU computing is the latency of the PCIe bus. Moving the 10,000×3 point array from Host (CPU) to Device (GPU) and fetching the resulting 381 MiB matrix back can easily outweigh the computational speedup.

Asynchronous Streams

In a production jet reconstruction pipeline, we would utilize `CUDA.stream()`. This allows the system to **overlap communication and computation**: while the GPU is processing Event N, the PCIe bus is simultaneously loading the data for Event N+1 into VRAM.

Unified Memory

For datasets of this scale, utilizing **Managed Memory** can simplify the code, though manual pre-fetching is often preferred to ensure the data is resident in VRAM before the kernel launches.

2. Memory Coalescing and SoA

GPUs achieve peak bandwidth only when threads in a Warp (a group of 32 threads) access contiguous memory addresses simultaneously.

The SoA Advantage

Our decision to transpose the input into a $3 \times N$ matrix for the CPU's SIMD units is directly applicable here. In a GPU kernel, this layout ensures **coalesced reads**, where the hardware combines 32 separate coordinate requests into a single high-speed memory transaction.

Caveat

Failing to use SoA results in **strided access**, which can degrade memory throughput by over **10×**, effectively idling the GPU's arithmetic units.

3. Shared Memory Tiling and Collaborative Loading

The Blocking/Tiling strategy used in the parallel CPU code maps directly to the GPU's **shared memory** - a tiny, ultra-fast L1 equivalent located on the GPU core.

Collaborative Loading

Rather than every thread fetching its own data from global VRAM, a **thread block** collaboratively loads a tile of points into shared memory.

Synchronization

By using a synchronization barrier (`sync_threads()`), we ensure the tile is fully loaded before all threads begin the O(N^2) distance computation within that local tile. This drastically reduces the pressure on the global memory controller.

4. Warp Divergence and Branchless Math

GPU hardware is designed for **lock-step execution**. If an `if/else` statement causes threads in a warp to take different paths, the hardware must execute them sequentially, a phenomenon known as **warp divergence**.

Numerical Stability

Though we got away with this one in my SIMD+Tiling CPU parallelization, in a production setting, `dx^2 + dy^2 + dz^2` can occasionally result in a tiny negative value (e.g., -10^-15) due to the high-speed **Fused Multiply-Add (FMA)** hardware on modern chips like the NVIDIA H100 or B200. We saw similar effects in the matrix generated by the parallel CPU code. Floating point rounding errors produce these small negative values.

To prevent NaN propagation, which is poisonous to the rest of the reconstruction pipeline, we must use branchless safety: `sqrt(max(0.0f0, dist_sq))`

Using `max` is a single hardware instruction that avoids the cost of a branch, keeping all 32 threads in the warp perfectly synchronized.

5. A Bonus Point: H100 and Beyond

For cutting-edge physics research at institutions like CERN, hardware such as the **NVIDIA H100 (Hopper)** or **B200 (Blackwell)** introduces additional architectural considerations.

HBM3e Bandwidth

These chips provide up to 4.8 TB/s of memory bandwidth. For 10,000 points, the bottleneck is rarely the arithmetic, it is register pressure. If a kernel uses too many local variables, the GPU cannot fit enough active warps on a core, leading to low **occupancy**.

TMA (Tensor Memory Accelerator)

Modern Hopper and Blackwell architectures include dedicated hardware (**TMA**) that handles collaborative memory loading. This allows compute units to focus purely on Euclidean distance calculations while specialized memory hardware manages the tile transfers.

Conclusion

Hence, an optimal GPU port is not just about running in parallel. It is about hardware saturation.
It is absolutely possible to implement a memory-saturated GPU setup capable of processing not just 100 million, but billions of events (in this case, datasets of 3D points in Euclidean space) per second by combining the above approaches in clever ways.

7. AI Usage Statement

In accordance with the exercise guidelines, I am including this statement regarding the use of AI.
AI tools (Google Gemini) were used to discuss performance engineering concepts such as hardware-level bottlenecks (e.g., L1 cache behavior, AoS vs. SoA memory layouts, SIMD vectorization limitations, and hyper-threading FPU contention). However, no coding assistants were used to generate the solution. The core architectural logic, including the task-flattening scheduler, the implementation of `@turbo`, the benchmarking pipeline, and the interpretation of the resulting hardware scaling data was entirely self-directed and hand-written to ensure a deep, verifiable understanding of high-performance Julia.